

CS 6004 Assignment-4 Beat The Interpreter

Object + Type sensitive points to analysis + Method Inlining

Team_OneM_oneU

Team ID: pa4008

Manish Kumar (24M0853)

Utkarsh Shetye (24M2122)

Contents

1	Optimization Description	1
1.1	Implementation Overview	1
1.2	Why 1Type 1Obj	1
1.3	Metric improved	1
2	Analysis Design: 1-Object-Sensitive + 1-Type Context	2
2.1	Context and Heap-Context Abstractions	2
2.2	Worklist Algorithm	3
2.3	Handling Virtual Dispatch (<code>handleDispatchCall</code>)	3
2.4	Assumptions	4
3	Transformation Details	4
3.1	Three-Way Call-Site Classification	4
3.2	Inlining Mechanism	6
3.3	Key Challenges Found/Addressed	7
4	Result tables	8
4.1	Functional correctness	8
4.2	Performance	8
4.2.1	Local Testcases	8
4.2.2	DaCapo Benchmark Results	9
5	Future enhancements	11
5.1	Improvement in precision for method with fewer objects	11
5.2	Conversion to analysis with generic type and obj	11
5.3	Handling of recursion in method inlining	11
5.4	Cost Based Inlining	11

1 Optimization Description

Virtual and interface dispatch in Java — the **invokevirtual** and **invokeinterface** bytecodes — are resolved at run time based on the dynamic type of the receiver object. This flexibility helps in achieving polymorphism, but it creates a challenge for static analysis and compiler optimization: without knowing which concrete method will be called at a given call site, a compiler cannot safely inline the call, and a static analysis tool must conservatively over-approximate the set to all possible callees.

The goal of this assignment is to implement a context-sensitive, points-to analysis for Java programs and reduce the virtual call and then inline it as much as possible.

1.1 Implementation Overview

We have implemented context-sensitive analysis using **1-Type-1-Object-Sensitive** points-to analysis with **1-object heap cloning** (1T1O + HC). Our implementation follows the framework established by Smaragdakis et al. (POPL 2011). This analysis later utilized for method inlining

The implementation consists of two phases:

- **Analysis Phase (ObjSensAnalysisTransformer):** A whole-program interprocedural analysis over Jimple IR. It classifies call sites as GLOBAL MONO, PER-CTX MONO, or TRULY POLY. Rest are treated as **unknown** means whose points to info can't be determined
- **Transformation Phase (InlineCheck):** Monomorphic sites are inlined directly. Per-context monomorphic sites are handled by cloning the enclosing method and redirecting calls before inlining. Truly polymorphic sites remain as dynamic dispatch. Unknown are simply ignored.

The net result is that virtual dispatch is eliminated from every call site where the analysis can prove, statically, that the receiver has at most one concrete type — either globally or within a particular calling context. Eliminated dispatch reduces: (a) the overhead of the invokevirtual / invokeinterface instruction, b) the callee's save and restore return address process when the body is small enough to benefit from inlining.

1.2 Why 1Type 1Obj

1 type 1 Obj is more precise than 1obj but slightly less precise than 2 obj. We approximate second context of 2 obj with 1 type. And as we see in paper pick your context well Smaragdakis et al. (POPL 2011). The paper claims that we can achieve nearly same precision of 2obj with 1Type1Obj. But at the same time 1Type 1Obj is easily scalable because the cost of 2 obj if there are C allocation site in each class then worst case it will be $O(C^2)$ whereas in case of 1 type 1obj it will be $O(C) * O(T)$ where C is the object created and T is different class type referred. This cost is much less than worst case cost of 2 obj. This makes 1 type 1 Obj as apt choice considering precision as well as scalability of analysis. Moreover function inlining depend more on class type so in this case 1 Type will be able to resolve maximum virtual calls.

1.3 Metric improved

Java programs make heavy use of dynamic dispatch, and even small programs can easily contain many virtual call sites. When running without JIT compilation (using the `-Xint` flag), each **invokevirtual** instruction results in a vtable lookup at runtime. This introduces a consistent overhead for every method call.

By resolving these calls statically and inlining them where possible, we eliminate this lookup entirely. In addition, the interpreter can directly execute the callee’s bytecodes without the overhead of crossing a method boundary, which reduces runtime cost.

To evaluate the impact of this optimization, we focus on two metrics:

- **Number of virtual call sites:** This gives a direct, instruction-level view of how much dynamic dispatch has been eliminated.
- **Wall-clock execution time (under `-Xint`):** This measures the end-to-end runtime improvement for each test case. Execution time is recorded in milliseconds and averaged over five runs to reduce noise.

Both metrics tend to improve together, since reducing the number of call sites lowers the interpreter’s workload, especially inside hot loops where dispatch overhead would otherwise accumulate.

2 Analysis Design: 1-Object-Sensitive + 1-Type Context

2.1 Context and Heap-Context Abstractions

To model calling behavior precisely without incurring the cost of full call-string tracking (which grows exponentially with call depth), the analysis uses two abstractions.

Calling Context (Context) A calling context is represented as a pair $[T : \text{ClassName}, O_\ell]$. Here, $T : \text{ClassName}$ denotes the class enclosing the allocation site that produced the caller of the receiver (note that this is not necessarily the allocated type). The term O_ℓ refers to the allocation statement (line number) of the receiver object. For the program entry point (`main`), we use a special empty context $[\text{null}, \text{null}]$.

Heap Context (HeapContext) Heap objects are tracked using a triple of the form $HC[\ell_1, \ell_2 \mid T : \text{AllocClass}]$. In this representation:

- ℓ_1 is the allocation site of the object itself,
- ℓ_2 corresponds to the allocation site of the caller’s receiver, which helps distinguish objects created in different calling situations,
- and $T : \text{AllocClass}$ stores the concrete type of the allocated object, which is later used during virtual dispatch.

We use two two function record and merge to create Heap context and context respectively, we maintain latest two elemen of record for heap context, In below record if second parameter is list then we will take first element ℓ' . so record will define as:

$$\text{record}(\ell, [\ell', C]) = [\ell, \ell']$$

When handling virtual calls, the callee’s context is derived by combining information from the receiver’s heap context and the current calling context:

$$\text{merge}(\ell, [\ell_1, \ell_2], c) = [T(\ell_2), \ell_1]$$

Here, $T(\ell_2)$ captures the type-based component of the context which is the type of caller’s allocation site, while ℓ_1 captures the object-sensitive component. This makes our approach as 1T1O design.

2.2 Worklist Algorithm

The analysis is implemented as an incremental, worklist-based fixpoint computation. Instead of re-processing everything repeatedly, it only revisits parts of the program when new information becomes available.

The main data structures used are:

- **localPT**: stores points-to information for local variables, indexed by method and context.
- **fieldPT**: maintains field-sensitive points-to information for heap objects.
- **returnPT**: aggregates possible return values for each method under a given context.
- **methodContexts**: records all contexts in which a method has been analyzed.
- **worklist**: a queue of (method, context) pairs that still need processing.

The fixpoint iteration proceeds as follows:

- **Skipping redundant work**: Before analyzing a (method, context) pair, the total number of known facts is compared with the count from the previous visit. If nothing has changed, the analysis skips this pair. This simple check avoids unnecessary recomputation.
- **Statement handling**: Each Jimple statement is dispatched to a dedicated handler depending on its type (e.g., object creation, field access, method calls, etc.). This keeps the implementation modular and easier to reason about.
- **Selective re-queuing**: After processing a call:
 - The callee is added back to the worklist only if its input (such as parameters or **this**) has gained new information.
 - Similarly, the caller is revisited only if the callee’s return values introduce new facts.

This ensures that the analysis focuses only on parts of the program where something has actually changed.

2.3 Handling Virtual Dispatch (`handleDispatchCall`)

For a virtual or interface call of the form $r.m(\text{args})$ inside method M under context C , the analysis proceeds as follows:

- First, it retrieves the points-to set of the receiver r under (M, C) . Each element in this set is a **HeapContext** that includes the concrete runtime type.
- For each such heap context h :
 - The target method is resolved by traversing the class hierarchy.
 - A new callee context is computed using the merge function:
 $\text{calleeCtx} = \text{merge}(\text{stmt}, h, C)$.
 - The receiver object (**this**) is then propagated into the callee under this new context.
- The arguments are propagated whenever any parameter context information has grown. The callee is reprocessed only if such growth occurs.

- Finally, the return values of the callee are propagated back to the left-hand side of the call. If this introduces new information, the caller is scheduled for reanalysis.

A key benefit of this design is that different receiver objects (allocated at different sites) lead to different callee contexts. As a result, their points-to information remains separate, avoiding unnecessary merging and improving precision.

2.4 Assumptions

The analysis is based on a few simplifying assumptions that help keep the design tractable while maintaining practical precision.

- **No reflection or dynamic class loading:** The analysis does not model objects created through reflection or dynamic loading mechanisms such as `Class.forName`, `Method.invoke`, or custom `ClassLoader` usage. Any call site that depends on reflective dispatch is conservatively classified as no points to info if no receiver object allocation is found.
- **No Phantom classes:** Phantom classes are conservatively ignored.
- **No Array allocation handling :** Array allocation are not handled.
- **No native methods:** Native method implementations are not analyzed. Their return values are assumed not to introduce new heap objects.
- **Single-threaded semantics:** The analysis assumes a single-threaded execution model. It does not consider interleavings, and field accesses are treated as non-volatile and non-synchronized.
- **No recursion:** Analysis assume program will not have recursion.

3 Transformation Details

3.1 Three-Way Call-Site Classification

After the points-to fixpoint converges, every virtual/interface call site in the user code is classified as follows:

Category	Condition	Action
GLOBAL MONO	Single target across ALL contexts	Inline unconditionally in place
PER-CTX MONO	Each context $\rightarrow$ exactly one target (targets may differ)	Clone enclosing method per target; redirect callers; inline within each clone
TRULY POLY	≥ 1 context sees multiple targets	Leave as dynamic dispatch (cannot inline)

Global Monomorphic (GLOBAL MONO): In the case of a virtual or interface call that always resolves to the same target, the transformation is a direct in-place devirtualization. We refer to this as Global Monomorphic. Since the target is unique globally, the call site is replaced by the actual body of the callee directly within the original caller’s code. This process is performed inside a loop to ensure that if the newly inlined body contains its own monomorphic call sites, they are also identified and expanded further. The transformation continues until the method reaches a stable state where no further inlinable targets remain.

Listing 1: Global Monomorphic Inlining

```

interface Operation {
    int execute(int x);
}

class Multiply implements Operation {
    public int execute(int x) { return x * 10; }
}

class Runner {
    static int run(Operation op, int val) {
        return op.execute(val); // call site
    }
}

public class Test {
    public static void main(String[] args) {
        Multiply m = new Multiply();

        int res = Runner.run(m, 1); // Globally invariant target detected for every
        // context inline directly with multiply body

        /* POST-TRANSFORMATION FINAL STATE:
         * Runner.run(m, 1) call
         * inlined directly into the main context.
         * int res = 1 * 10;
         */
    }
}

```

Per-Context Monomorphic (PER-CTX MONO): The transformation is applied based on the category, as shown in the table. In the case of PER-CTX MONO (Per Context Monomorphic), a call site is monomorphic within the specific context. For such cases, a specialized clone of the original caller method for each unique target is created. Inside each clone, the call site is inlined with its corresponding monomorphic target. Finally, all invocations of the original caller are rewritten to call the specialized version.

Listing 2: Per-Context Monomorphic Inlining Example

```

interface Operation {
    int execute(int x);
}

class Multiply implements Operation {
    public int execute(int x) { return x * 10; }
}

class AddOp implements Operation {
    public int execute(int x) { return x + 10; }
}

class Runner {
    static int run(Operation op, int val) {
        return op.execute(val); // call site
    }
}

public class Test {
    public static void main(String[] args) {
        Multiply m = new Multiply();
        AddOp a = new AddOp();

        /* Context 1: Runner.run is specialized for Multiply.
        Redirected to: Runner.run_Multiply_L25 */
        int ans1 = Runner.run(m, 1);

        /* Context 2: Runner.run is specialized for AddOp.
        Redirected to: Runner.run_AddOp_L26 */
        int ans2 = Runner.run(a, 1);

        /* POST-TRANSFORMATION FINAL STATE:
        * int ans1 = 1 * 10
        * int ans2 = 1 + 10;
        */
    }
}

```

Truly Polymorphic (TRULY POLY): For Truly Polymorphic (TRULY POLY) calls, the analysis determines that the dispatch cannot be simplified into a direct call because the receiver of the method resolves to multiple possible target methods within a single context. Since there is no unique target to replace the call site with, inlining is not possible, and these sites are ignored by the transformation.

3.2 Inlining Mechanism

The inlining process integrates the callee’s method body into the caller while preserving semantic correctness through the following steps:

- **Namespace Isolation:** To prevent callee local variables from interfering with the caller, we utilize Soot’s `DefaultLocalGenerator` to generate unique local variables within the caller’s body for every variable in the callee.
- **Identity and Parameter Binding:** Jimple `IdentityStmt` instructions (for parameters and the `this` reference) are transformed into standard `AssignStmt` units. These links the call site’s actual arguments and the receiver object directly to the newly created local variables.

- **CFG and Jump:** To maintain the Control Flow Graph, we use the `redirectJumpsToThisTo()` utility. This automatically updates all jump targets within the callee so they point to the correct instructions after being moved to the caller’s body.
- **Result Propagation and Injection:** `ReturnStmt` instructions are converted into assignments to the caller’s result variable. The final modified statement block is then atomically injected into the calling context using the `insertBefore()` method.

3.3 Key Challenges Found/Addressed

- **Analysis termination point:** The analysis terminates properly in the normal program, but a major challenge is faced during running with benchmarking. We end up in a situation where, if we keep including method calls, benchmarking may not terminate. So we keep a `MAX_CONTEXTS_PER_METHOD` flag to limit context within a method, and a `MAX_TOTAL_CONTEXTS` flag to limit the total number of contexts across the entire program. This helps us to terminate analysis, and the solution is sound. Although it might not be fully precise with benchmarks because benchmark programs have many function calls, after a certain number of contexts, we decided to skip analyzing method calls. To improve the precision, we keep CHA as a fallback option for methods that were not analyzed. This helps us increase precision and at the same time keep the analysis sound.
- **Benchmark incompatibility with JDK version > 8:** Benchmarks like `fop` and `luindex`, etc., use methods that have known issues, resulting in exceptions. To resolve this, we excluded the body of library calls that are having problems. This helps us run the analysis for these benchmarks.
- **Maintaining Control Flow:** Moving code from one method to another often breaks original jump labels of callee and branches. To resolve this, we used Soot’s redirection tools (`redirectJumpsToThisTo`) and `NOP` placeholders, ensuring all internal `goto` logic remains valid after the code is injected.
- **Transitioning Returns:** We handle the callee’s exit points by mapping them to safe transitions within the caller’s body, so that program’s control flow remains intact after the original call site is removed.
 - **Value Mapping:** `ReturnStmt` is transformed into an assignment to the caller’s target variable.
 - **Exit from Method:** For `void` methods returns are replaced with `NOP` statements to preserve internal jump targets and CFG integrity.
- **Local Variable Remapping:** To prevent namespace collisions, Soot’s `DefaultLocalGenerator` creates unique locals for the caller’s scope. A mapping table is then used to substitute all original callee references with these newly generated variables
- **Fixed Point Inlining (Nested Calls):** Single-pass inlining would miss opportunities when an inlined method contains its own monomorphic calls. To solve this, a fixed-point iterative loop that continues to expand call sites until no further inlining opportunities are possible.
- **Per Context Monomorphic before Global Monomorphic:** Global inlining is an in-place mutation that permanently modifies a method’s body. If performed before specialized cloning, these changes would propagate across all contexts, destroying analysis isolation. We resolved this by executing a `clonePass()` first to isolate method bodies across each context before the inlining phase begins, ensuring all subsequent modifications remain local to their specialized versions.

4 Result tables

4.1 Functional correctness

Functional correctness was verified by comparing the standard output of the original programs against the Soot-transformed versions. Table 1 summarizes the specific features validated for each test case.

Table 1: Functional Correctness Verification

Test	Functionality Verified	Status
Test1	Global monomorphic call inlining and semantic preservation	Pass
Test2	Polymorphic branching logic and call site differentiation	Pass
Test3	Mix of Per Context monomorphism and Polymorphic call with inline decision	Pass
Test4	Per-context monomorphic call resolution with 2 levels	Pass
Test5	Multi target inlining via method cloning with single target called twice	Pass
Test6	Complex control flow with multiple nested return paths (static calls)	Pass
Test7	Support for deep static call chains (10 levels) + Aggressive Inlining	Pass
Test8	Virtual call chain resolution with instance method	Pass
Test9	Allocation site awareness and reallocation inside loop	Pass
Test10	Alias awareness, deep field chains, and global monomorphism	Pass

4.2 Performance

Performance measures play a key role in identifying improvements over the existing program. Our main heuristic for performance measurement is **number of virtual calls resolved and inline successfully** and improvement on **wall-clock execution time (under -Xint)**

We have measured performance against the Dacapo benchmark and our application test cases.

As the Dacapo benchmark was built in JDK 8, we excluded some library calls that are causing exceptions and are not compatible with the latest JDK version.

We broadly categorize performance analysis into two categories: **With library inclusion** and **Without library inclusion** (in this, we are only excluding known JDK and Java libraries, not any other libraries, so if the benchmark class uses any built-in libraries, they are not ignored). As in each of these categories, the no of method calls exceeding our threshold of `MAX.CONTEXTS.PER.METHOD` flag and `MAX.TOTAL.CONTEXTS` flag. So we started safely ignoring the method context once we reached the threshold. As a result, the resolution rate will be lower because the total count will reflect the actual number of contexts generated.

We further analyze both categories using CHA for the method context, which are ignored because they are called after the analysis threshold is exceeded. This increases the resolution rate by analyzing the ignored methods identified by CHA.

4.2.1 Local Testcases

Each test case was executed five times using the `-Xint` flag to calculate average execution times. Table 2 summarizes the execution times before and after optimizations, along with the respective speedup and

percentage of improvement. Every test case includes critical logic within a loop to ensure measurable and meaningful performance outcomes.

Table 2: Performance Analysis of Local Test Cases in time(Average of 5 Runs in milliseconds)

TestCase	Avg Before (ms)	Avg After (ms)	Speedup	Improvement (%)
Test1	73.8	51.8	1.42x	29.81%
Test2	36.6	37.4	0.98x	-2.19%
Test3	146.4	81.2	1.80x	44.54%
Test4	271.0	80.8	3.35x	70.18%
Test5	1120.4	695.6	1.61x	37.91%
Test6	369.4	153.2	2.41x	58.53%
Test7	544.8	124.0	4.39x	77.24%
Test8	2975.0	1608.2	1.85x	45.94%
Test9	318.2	256.0	1.24x	19.55%
Test10	2401.6	2005.2	1.20x	16.51%

The performance analysis indicates that the most significant speedups, such as the 4.39x (77.24%) gain in Test 7, occur when multiple levels (10 levels) of aggressive inlining effectively eliminate method dispatch overhead. In contrast, polymorphic scenarios like Test 2 show no major (negative) improvement, as inlining is not done in such cases. Test4 demonstrates a per context monomorphic scenario, where the analysis differentiates call sites based on their specific contexts and performs specialized method inlining, eliminating virtual dispatch overhead significantly. Across the remaining test cases, the transformation achieved a stable and significant improvement ranging from 16% to 60%.

Table 3 summarizes the virtual call resolution after applying **1 type 1 obj** context sensitivity analyses on user defined test cases

Table 3: Performance Analysis of Local Test Cases with virtual call resolution

TestCase	Total virtual calls	Calls Resolved(1Type 1Obj)	Resolution Rate (%)
Test1	1	1	100.0%
Test2	1	0	0.0%
Test3	3	2	66.7%
Test4	3	3	100.0%
Test5	4	4	100.0%
Test6	0	0	0.0%
Test7	0	0	0.0%
Test8	7	7	100.0%
Test9	3	2	66.7%
Test10	3	1	33.3%

Test 6 handles inlining of static calls with complex flow and multiple returns and Test 7 handles inlining of long call chain of static calls. As these two examples are focus on showing performance with respect to inlining of static call and we have seen in 2 that it gives significant speedup on inlining them.

4.2.2 DaCapo Benchmark Results

The benchmarks were evaluated using the following configurations:

- **lib_cha**: Whole-program analysis (library inclusion) with CHA fallback.

- **lib**: Whole-program analysis without CHA fallback.
- **cha**: Application only analysis with CHA fallback.
- **none**: Baseline application-only analysis with no fallback.

The result of the analysis is shown in Table 4.

Table 4: Benchmark Analysis Performance

Benchmark	Config	Total Calls	Resolved	Res. Rate	Analysis Time (s)
avroa	lib_cha	67,299	56,912	84.6%	410.89
	lib	67,299	9,411	14.0%	405.90
	cha	2,544	2,155	84.7%	27.24
	none	2,544	654	25.7%	27.64
batik	lib_cha	133,452	107,031	80.2%	811.24
	lib	133,452	14,732	11.0%	773.60
	cha	9,030	7,173	79.4%	390.72
	none	9,030	2,236	24.8%	390.89
fop	lib_cha	147,852	121,575	82.2%	730.56
	lib	147,852	19,234	13.0%	749.65
	cha	15,666	13,223	84.4%	203.31
	none	15,666	2,738	17.5%	209.48
luindex	lib_cha	67,849	57,098	84.2%	486.06
	lib	67,849	9,439	13.9%	488.15
	cha	2,964	2,418	81.6%	9.01
	none	2,964	852	28.7%	8.51
xalan	lib_cha	78,060	66,295	84.9%	389.23
	lib	78,060	10,040	12.9%	393.29
	cha	10,352	8,867	85.7%	436.81
	none	10,352	1,611	15.6%	450.67

We define the **Resolution Rate** as the percentage of virtual call sites resolved to a single target. A call site is considered unresolved if it is either: (i) **Truly Polymorphic**, where multiple receiver types prevent devirtualization, or (ii) **Unknown**, where no points-to information is available. Both cases are counted as unresolved, while monomorphic targets (global or per-context) are counted as resolved. The results show that falling back to CHA improves the resolution rate for unresolved (unknown and polymorphic case).

The original lib (without cha) has less resolution rate than lib_cha, the reason behind this is that we have put a cap on MAX_CONTEXT_PER_METHOD and MAX_TOTAL_CONTEXTS, this step is taken as in benchmark there are lots of methods if we not put a cap then methods call keep on happening for few hours or may result into out of memory if method calls are too aggressively happening so to terminate analysis within approx 12 min for a benchmark this cap is added. After adding cha as fallback for ignored method call, we are able to resolve maximum cases within less time. Same applies for without lib also i.e. none(it doesn't include normal java, sun, org packages but it will include other libraries used by benchmarks. So if we add cha here for ignored method calls, then again resolution rate improved).

5 Future enhancements

5.1 Improvement in precision for method with fewer objects

By counting no of 2obj objects and switching between 1Type1obj and 2Obj based on a threshold count. This will provide high precision for the method with less object context and, at the same time, help maintain scalability as the number of object contexts exceeds a limit.

5.2 Conversion to analysis with generic type and obj

We can extend it by taking a command line argument for obj and type, and accordingly implement the analysis. This implementation can easily be extended by using this approach to a generic object sensitive analysis

5.3 Handling of recursion in method inlining

Recursion is a costly call and if we are able to inline some of these calls, it will improve execution times a lot.

5.4 Cost Based Inlining

Cases where doing aggressive inlining is not always beneficial. So, we avoid doing aggressive inlining when the cost of doing it is higher.